

PropCov: Effective Coverage Reporting for Property-Based Testing

JESSE COULTAS, University of Illinois Chicago, USA

JOSEPH WISEMAN, University of Illinois Chicago, USA

LUÍS PINA, University of Illinois Chicago, USA

Property-based testing (PBT), introduced by Haskell’s Quickcheck, is becoming more popular with successful ports for other languages, such as Java’s `junit-quickcheck`. With PBT, developers write a *property test* and a *data generator*. The *data generator* takes a source of non-determinism and uses it to output well-formed data. The *property test* exercises the System Under Test (SUT) using the random well-formed data from the generator to ensure a particular property always holds (e.g., data serialized and deserialized should be equal to the original data). The PBT framework then performs many *trials*, each generating fresh data and executing the property test. A test failure shows a bug to developers, typically in edge-cases. A passing test gives some assurance on the quality of the SUT with regards to the property being tested. Unfortunately, well-known test coverage tools that are instrumental for understanding unit testing work poorly for PBT.

In this paper, we present PropCov, a tool for understanding statement coverage in PBT that also provides suggestions for coverage improvement. PropCov employs a novel combination of static analysis with PBT to approximate the maximum possible statement coverage, providing an effective measure of the PBT coverage and making suggestions to developers of where to improve existing tests. PropCov features an easily extensible architecture designed to support new languages, build systems, and PBT frameworks. We evaluated PropCov using 25 Java projects using `junit-quickcheck` or `jqwik`, totaling 293 properties, and found that existing tools report missed coverage that is impossible to reach (86% of lines that JACoCo reports as not covered), which leads developers to consider hundreds of extra lines of code (2897). Unlike existing coverage tools, PropCov results are accurate — only 6.4% of all properties contain unfeasible code, and PropCov only misses 3% of reachable code. Using PropCov’s suggestions, we increased the coverage of 42 tests over 7 projects and found 5 new bugs in 4 projects.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; *Empirical software validation*.

Additional Key Words and Phrases: property-based testing, code coverage, statement coverage, static analysis, call graphs, Java, test quality

ACM Reference Format:

Jesse Coultas, Joseph Wiseman, and Luís Pina. 2026. PropCov: Effective Coverage Reporting for Property-Based Testing. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA037 (October 2026), 24 pages. <https://doi.org/10.1145/3832128>

1 Introduction

Property-based testing (PBT) allows developers to express sophisticated properties that should always hold true. For instance, Figure 1 shows a *round-trip property* implemented in Java with `junit-quickcheck` that applies to libraries that serialize and de-serialize data, and states that *any*

Authors’ Contact Information: Jesse Coultas, University of Illinois Chicago, Chicago, USA, jcoult1s@uic.edu; Joseph Wiseman, University of Illinois Chicago, Chicago, USA, jwisem6@uic.edu; Luís Pina, University of Illinois Chicago, Chicago, USA, luispina@uic.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA037

<https://doi.org/10.1145/3832128>

```

01: interface Serial<T> { byte[] serialize(T t); T deserialize(byte[]); }
02:
03: @Property void roundtrip(@From(ListGen.class) List target) {
04:     Serial<List<Integer>> s = new IntListSerial();
05:     assertEquals(target, s.deserialize(s.serialize(target))); }
06:
07: List ListGen.generate(SourceOfRandomness r, GenerationStatus s) {
08:     List ret = new LinkedList();
09:     for (int i = 0 ; i < r.nextInt(100) ; i++) ret.add(r.nextInt(1_000_000));
10:     return ret; }

```

Fig. 1. Example of a test for property (roundtrip) and associated data generator (ListGen.generate). Figure 2 shows implementations of interface Serial. Argument *r* is a random-number generator, and *s* is an (unused) object that can keep state between calls to generate. The generator outputs a list of random size with random contents (Line 10). The test then serializes and deserializes each generated list, testing if the property holds by checking whether the result is the same as the original list (Line 5).

data serialized and then de-serialized must be the same (Line 5). A PBT framework runs many *trials* of the same property (e.g., 100 is a common default), each with fresh data from a *generator* that takes a source of randomness and creates concrete data — lists with random integers and random size (Line 9). A trial that finds a violation can be later replicated using the same generated data.

Unfortunately, a property test that does not find any violations gives developers a false sense of security in thinking that the System Under Test (SUT) implements the property correctly. In fact, the property test may be unable to find problems because either: (1) the generator lacks diversity (e.g., only generating large numbers), or (2) the property is too narrow and actually does not cover the expected functionality (e.g., a different property focused on list concatenation never covers empty lists). Developers often identify PBT’s opaqueness as a barrier to adoption, comparing PBT poorly against (unrealistic) exhaustive tests that “*hit things I actually care about*” [9], and mentioning unknown “*holes in the testing coverage*” [13]. A recent study on developers using PBT [36, 37] also reports on developers struggling to trust PBT that does not find any issues, and resorting to: injecting bugs, manual analysis of the generated data, and ad-hoc analysis of coverage reports.

Making matters worse, well established techniques to measure and improve commonly used unit testing fail with PBT. One such technique is code coverage (e.g., using JACoCo): run a test suite with code coverage enabled and write new tests for uncovered code. Writing new property tests to reach particular lines is misguided, because a suite that covers line *L* with one test *T*₁ but misses the same line *L* with another test *T*₂ is still incomplete, as *L* may be critical to finding an error with *T*₂ even though *T*₁ already covers it.

In this paper, we present a technique that makes commonly used and well understood statement coverage meaningful for PBT. Our **first novel insight** is to consider each individual property test in isolation. Unfortunately, when applied to a single test, existing code coverage techniques include a large amount of uncovered code that is actually impossible to reach. We show in Section 4.1.3 that the vast majority of such JACoCo reports is impossible to be reached from the test (86% lines over 85% methods), leading developers to consider an average of 2897 unfeasible lines over 693 methods. Our **second novel insight** is to restrict coverage reporting for each property test to only code that can be reached from that property test. Note that it is challenging to find which code can be reached from a particular test, especially in imperative languages that keep a large amount of mutable state and use dynamic function calls (e.g., method dispatch based on the runtime type of the receiver object). We use existing state-of-the-art techniques to build a call-graph that approximates the maximum theoretical statement coverage for each property test, augment that

call-graph with coverage information observed when running the property test, and present it as a report to the developer. The amount of uncovered code in our report provides the developer with a concrete measure of how complete each property test is, and where to focus efforts to improve it.

We implement our technique as an extensible framework that targets JVM languages (e.g., Java, Scala, Kotlin, Groovy) and respective PBT frameworks (e.g., `junit-quickcheck` [43], `jqwik` [1]). Our tool, named `PropCov`,¹ leverages state-of-the-art analysis framework `OPAL` [30, 31, 40] to deal with dynamic language features — which concrete methods can be reached via dynamic method calls, and targets of `invokedynamic` [62] (e.g., lambda functions and stream operations in Java). We describe the design and implementation of `PropCov` for Java together with an extensive experimental evaluation using 293 properties over 25 projects. `PropCov` results are accurate — only 6.4% of all reports contain unfeasible code (Section 4.1.2), and `PropCov` only misses 3% of reachable code (Section 4.1.1). `PropCov` uses a novel heuristic (Section 3.4) that scores program paths according to how close they were to be reached during execution, and suggests program points to improve. Following `PropCov`'s recommendations, we improved the coverage of 42 properties and found 5 bugs, all on projects unfamiliar to us (Section 4.2.3). At the time of writing, 2 bugs were confirmed and fixed by the developers, and we submitted 9 pull requests, 5 merged by the developers (other bugs/PRs are still pending). `PropCov` also allows to perform **novel scientific studies** on PBT in general. Using `PropCov` and our corpus of projects, we measure the overall coverage of existing PBT as 72%/74% of lines/methods, as opposed to 16%/20% reported by `JaCoCo` (Section 4.2). We also measure the role of the number of trials in PBT (Section 4.2.2), and recommend a lower default, 10 instead of 100, together with using `PropCov`'s reporting to understand when to increase it.

In summary, this paper makes the following contributions:

- A novel coverage mechanism specifically designed for PBT based on a static call-graph and dynamic code coverage, comparing their theoretical maximum coverage with the observed coverage during a test run.
- The modular design of `PropCov`, a novel tool that is designed to support any language and PBT framework that targets JVM bytecode (e.g., Java, Scala, Kotlin, Groovy).
- The implementation of `PropCov` for Java, together with an extensive evaluation using 293 properties over 25 Java projects. In the process, we improved the coverage of 42 tests over 7 projects, and found 5 previously unknown bugs. We opened 9 pull requests, with 5 merged at the time of writing, and opened 2 issues, both confirmed and fixed by the developers. The remaining pull requests remain open.
- A novel study on existing PBT for Java, not possible without `PropCov`, that measures PBT test coverage as 72%/74%, and suggests lowering the default number of trials to 10.
- A replication artifact that allows future researchers to reproduce the results in this paper, and to build on `PropCov`. Both the source code of `PropCov`'s prototype [28] and the replication artifact [27] are available online.

2 Property-Based Testing and its limitations

To introduce property tests, let us consider the example property specified in Lines 3–5 of Figure 1 which uses `junit-quickcheck` [43]. Such a *round-trip property* is common for serialization libraries and states that serializing and deserializing any valid data (provided via argument `target` on Line 3) should result in the original data (Line 5).

PBT frameworks execute the same property test many times, and we denote each execution as a *trial*. For each trial, the PBT framework runs the test with fresh random data in an effort to find bugs

¹Short for **Property Coverage**

via input diversity, ideally testing corner-cases that are hard to anticipate by the developer. In our example the property test expects a well-formed list. PBT allows developers to write *generators* to bridge the gap between easy-to-generate random data and well-formed input data. Our example has such a generator (Lines 7 to 10) that takes a random number generator (argument *r*) and uses it to generate a well-formed list with random size and random contents (Line 9). When PBT frameworks find an input that violates any assertion in a test, they report the generated inputs together with the seed used to generate that particular input. Later, developers can use that information to trigger the same bug reliably, assuming test correctness depends only on the arguments passed to it (*i.e.*, tests are not flaky [50]).

However, **what happens when PBT cannot find any bugs?** There are three options that explain such behavior. **First**, the *System Under Test* (SUT) implements the property correctly. Of course, testing cannot guarantee the absence of bugs [16], so developers must be careful when considering this option. **Second**, the generated input is not diverse enough to find existing bugs. Unfortunately, as we report in Section 4.2.2, simply increasing the number of trials cannot increase diversity enough to trigger existing bugs. **Third**, the test is *too narrow* and cannot reach code that developers expect to cover. Unfortunately, each option is indistinguishable from the others. Developers often complain about PBT’s opaqueness [9, 13, 36, 37].

2.1 Existing coverage tools and PBT

Figure 2 shows a possible implementation of a serialization library that supports lists, which can be tested by the property test in Figure 1. Consider that there is a bug in Line 26 that serializes small integers that fit in two bytes as a character. For example, negative integers lose the sign when serialized, and are then deserialized incorrectly as positive. Also, the implementation of the `IntListSerialized` is more complex than just the code shown. It uses many small methods to encapsulate and reuse behavior, exposing state safely to client classes, thus following good general practices of software engineering. The example abstracts such unrelated methods as `m1` through `m100`, which stand-in for unrelated code, that our experimental evaluation reports as 693 methods on average (Table 2 in Section 4.1).

Note that the property test in Figure 1 *can* find this bug. However, each trial generating an integer in the range $0-2^{31}$ has a very low chance of generating an integer in the small range 8–256 in the 100 trials that PBT frameworks may use as default. In practice, the property test does not find this bug and the test always passes. This bug is adapted from a real bug in project *manu*, which we found when addressing a *PROPCOV* alert that not all serializer classes were reached, and addressed by ensuring the generator outputs data fitting each possible class.

Developers typically use code coverage tools, such as JACoCo [42], to understand how tests exercise the SUT. Using such a tool on a suite of property tests may not reveal this particular bug, as another property test may cover Line 26 but not reveal the bug (*e.g.*, ensuring characters with accents are serialized correctly). It thus makes sense to focus on each individual property test. Unfortunately, JACoCo considers all methods in the class (and all methods in all other classes loaded during the test execution), including unrelated methods `m1` through `m100`. A developer worried about the example property test we are following never failing is thus left with a coverage report that contains a very large number of *unfeasible methods* — methods that cannot possibly be reached by the property. In Section 4.1.3, we show that most of the code in JACoCo’s reports (86% lines over 85% methods) is unfeasible. Even though the report does state that Line 26 and respective branch are not covered, it is very hard for the developer to notice it among all the code listed in the coverage report.

We note that the code in Figures 1 and 2 is based on a real property written for the industrial project *mph-table* [44], and it is thus representative of how practitioners actually use PBT. We

```

11: class IntListSerial implements Serial<List<Integer>> {
12:   byte[] serialize(List l) {
13:     Serial<Integer> bs = new ByteSerializer(), sS = new ShortSerializer(),
14:     cs = new CharSerializer(), iS = new IntSerializer();
15:     byte[] ret = new byte[0];
16:     for (Integer i : l) {
17:       Serial<Integer> s;
18:       if (i<8) s=bs; else if (i<0xFF) s=cs; else if (i<0xFFFF) s=sS; else s=iS;
19:       ret += s.serialize(i); } // end for
20:     return ret; } // end method serialize
21:   ...
22:   void m1() { ... } ... void m100() { ... } } // end class IntListSerial
23: class ByteSerializer implements Serial<Integer> { ... }
24: class ShortSerializer implements Serial<Integer> { ... }
25: class IntSerializer implements Serial<Integer> { ... }
26: class CharSerializer implements Serial<Integer> { byte[] serialize(Char i) { /* BUG */ } ... }

```

Fig. 2. Example Java program called by a property that leads to large reports, that include unfeasible methods m1 through m100, when using typical code coverage tools. For Line 5 in Figure 1 to fail, Line 9 needs to output a number in a small range that fits in a char but not in a short or byte, which is very unlikely (chance of 0.02%) and thus leads to the test passing when it should fail. For compactness, we use the operator += on Line 19 to concatenate byte arrays and elided overrides for methods serialize/deserialize in classes ByteSerializer, ShortSerializer, IntSerializer, and CharSerializer.

adapted the code to include a bug from project manu [29] to also show how easy it is to miss potential bugs without proper coverage report. mph-table uses a virtual call to dispatch to different serializers (Line 19). manu has the bug described (Line 26), but dispatches using a cascading if-else.

3 PROPCOV

PROPCOV is a novel approach for measuring coverage for PBT, which we developed with the main goal of solving the problems identified in Section 2. Another goal of PROPCOV’s design is to be easily adaptable to other languages and PBT frameworks, based on JVM bytecode. As such, PROPCOV follows a modular architecture based on *extensions* for each source code repository, build system, and PBT framework. Figure 3 shows an overview of PROPCOV’s architecture.

First, PROPCOV takes as input the *System Under Test (SUT)* and a *configuration* that defines a collection of property tests that are part of a project. PROPCOV starts by using the first extension — *source* — to acquire the source code of the project specified. Currently, PROPCOV supports local projects and Git repositories. Then, PROPCOV uses the second extension — *build* — to turn the source code into JVM bytecode. Currently, PROPCOV supports Java projects built with Maven [17]. After building a project, PROPCOV uses the next extension — *test* — to interact with the particular PBT framework that the project uses. Our current prototype supports junit-quickcheck [43] and jqwik [1]. We designed PROPCOV with a modular architecture to be extensible to new build systems and languages (such as Scala built with sbt, or Java/Kotlin built with Gradle) and more PBT frameworks (such as scalacheck [54] and americium [53]).

PROPCOV’s next step is to approximate the theoretical maximum coverage of the property using static analysis, resulting in a feasible callgraph. PROPCOV’s prototype currently supports Opal [30, 31, 40], and its modular design allows future users to add different analyses, such as Soot [67]. Note that Opal already supports many analyses, which we describe in Section 3.2.

Unfortunately, we found that the feasible callgraph still contains inaccuracies due to common code patterns, primarily due to exception propagation and use of iterators/iterable objects. PROPCOV

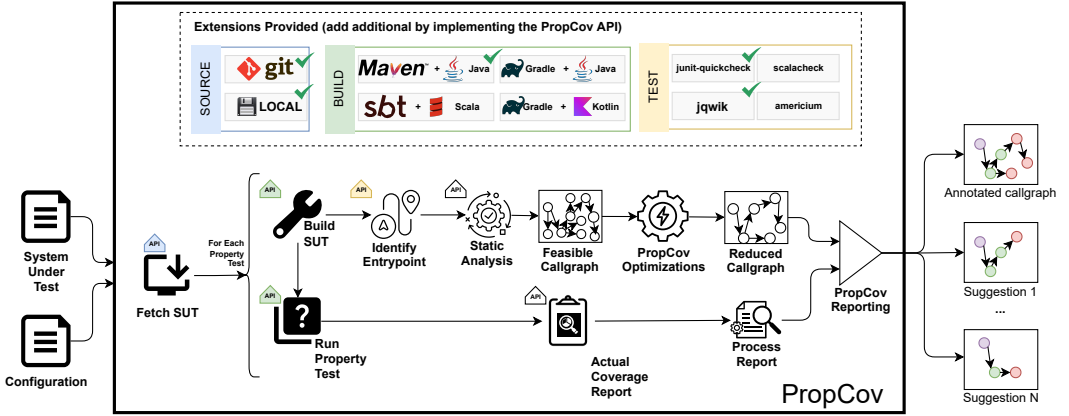


Fig. 3. Overview of PROPCov’s architecture. We note that PROPCov is modular to enable supporting any build system, language, PBT framework, static analysis, and coverage tool that targets JVM bytecode. The green checkmarks denote what PROPCov’s prototype supports. PROPCov currently supports Opal [30, 31, 40] and JACoCo [42], as the static analysis and coverage tools, respectively (not shown).

optimizes the resulting callgraph to remove such inaccuracies, further reducing the size of the callgraph.

Finally, PROPCov executes the property and uses another extension to obtain a coverage report. PROPCov’s current prototype supports the de-facto tool for Java coverage JACoCo [42]. Then, PROPCov combines the analysis results with the actual coverage observed, in the form of an annotated callgraph with suggestions of which paths seem to yield the best improvement if covered.

As the end result, PROPCov presents developers with a callgraph very similar to the examples depicted in Figure 4 (depending on the analysis used, as explained in Section 3.2), with certain high value paths highlighted according to PROPCov’s heuristic (explained in Section 3.4, not shown in Figure 4).

3.1 Writing Extensions

PROPCov is designed to be extensible by writing new extensions, thus supporting additional version control tools (e.g., SVN), build systems (e.g., Gradle, sbt), programming languages (e.g., Scala, Kotlin), PBT frameworks (e.g., ScalaCheck, Americium), static analyses (e.g., Soot), and coverage tools. The current prototype supports: git repositories and local sources, maven, Java, junit-quickscheck and jqwik, Opal, and JACoCo. Note that we designed PROPCov with projects that target JVM bytecode in mind.

To write an extension, PROPCov users need to extend a Java abstract class and provide the following functionality in separate methods: retrieving the **source**, **building** the retrieved source and running a particular test, identifying which methods are **tests**, performing **analysis** on the SUT to build a suitable callgraph given a property test as entry-point, and measuring the **coverage** when executing a particular property test.

We note that the *build* extension should use *coverage* tool when building the project. In our case, it means that we needed to modify some build scripts to include support for JACoCo. Such effort was straightforward and mostly copy/pasting the JACoCo configuration into each Maven build file.

3.2 Static Analysis

The first step in PropCov is to perform a static analysis on the SUT combined with the property, and custom generator if one exists. The goal of the analysis step is to generate a callgraph that represents the maximum feasible coverage possible for the property under analysis. Although most Java code (and JVM bytecode) is straightforward to analyze, there are some challenging language features, most notably: virtual method invocation, and lambdas. Figure 4 shows why virtual method invocation is challenging, inspired by the examples we have been following. Note that Line 5 in Figure 1 invokes the interface method `serialize`, with no body, defined in Line 1. Figure 2 shows 5 possible concrete implementations of method `serialize`: 2 explicitly for classes `IntListSerial` and `CharSerializer` (Lines 12 and 26), and 3 implicit implementations for each other class listed (Lines 23–25). The challenge for a static analysis building the callgraph is to **resolve the method**: choose which call is feasible from each call site. For instance, the only feasible concrete implementation of method `serialize` reachable from Line 5 is `IntListSerial`. However, Line 19 can reach all other concrete implementations *except* `IntListSerial`.

3.2.1 Supported Analyses. PropCov can be configured with many method resolution algorithms as implemented by the OPAL JVM bytecode analysis framework [30, 31, 40]: CHA, RTA, and k-CFA. **Class Hierarchy Analysis (CHA)** provides the simplest possible solution and considers all known concrete implementations as feasible. Figure 4a shows it always adds all known concrete implementations of each virtual method, including unfeasible methods such as reaching `IntListSerial.serialize` from Line 19. **Rapid Type Analysis (RTA)** tracks class instantiation and considers only concrete classes observed to create new objects. Figure 4b shows that RTA accurately resolves the virtual call on Line 16 because it can track the instantiation of the list on Line 8. However, RTA can behave as poorly as CHA when there are many possible instantiations, as Figure 4b shows for the call to `serialize` on Line 5 and Line 19. **k-CFA** perform sophisticated points-to analysis with increasing context sensitivity. OPAL provides support for 0-CFA and 1-CFA, and these are the most accurate algorithms [61]. Figure 4c shows that k-CFA, for this particular example, is able to resolve the call to `serialize` on Line 5 to the only feasible implementation in `IntListSerial`, thus resulting in the highest accuracy.

3.2.2 Accurate support for custom generators. The ability for developers to write custom data generators is a powerful feature of PBT. Figure 1 shows such a generator on Lines 7–10, which generates inputs passed to the property test via the argument `target` on Line 3. Custom generators are not limited to primitive types and Java collections as shown in the example. In fact, custom generators can generate complex custom types defined in the SUT. Given that it is the PBT framework itself (e.g., `junit-quickcheck`) that calls the generator, there is no direct connection between the two in the code that OPAL analyzes, which in turn leads to poor quality results for RTA and k-CFA.

When starting from the property test (Line 3), the OPAL’s analyses do not see any instantiation of the type `List`, which causes them to assume that argument `target` is `null`. As a result, such analyses consider that the body of the loop on Line 16 is not reachable, and thus miss the important edge from `IntListSerial.serialize` to `s.serialize` on Line 19. We observed this exact issue on project `capsyl` [56], which defines a property to check if two UUIDs are the same, together with a custom UUID generator. When analyzing such property test, which takes UUIDs as arguments, both RTA and k-CFA fail to connect those arguments with the PBT framework code that runs the custom generator to instantiate them. As a result, RTA and k-CFA assume the UUID arguments to be `null` and terminate the analysis prematurely at the first use of such arguments. Note that solving this problem is not as straightforward as simply adding the code of the PBT framework itself to OPAL’s analyses, as such code relies on reflection which PropCov currently does not support.

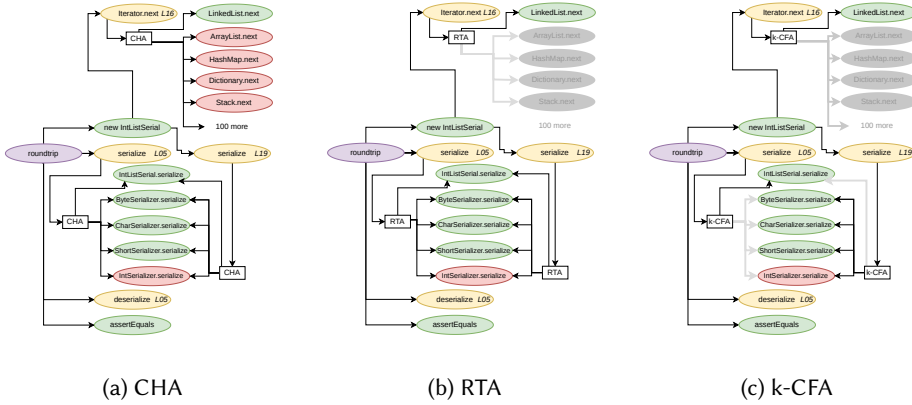


Fig. 4. Possible callgraphs with different resolutions for virtual calls, using different static analysis algorithms, for the code shown in Figures 1 and 2. CHA (4a) considers as valid all possible concrete methods (including `IntListSerial` on Line 19). RTA (4b) resolves the virtual call to `Iterator.next` on Line 16, but behaves as poorly as CHA for the call to `serialize` on Lines 5 and 19. k-CFA (4c) produces a minimal feasible callgraph. Nodes represent methods, and the colors mean: test nodes (purple), covered nodes (shades of green, according to coverage), uncovered nodes (red), and virtual method nodes (yellow).

PROPCOV supports custom generators by configuring OPAL’s RTA and k-CFA analyses to “assume” that certain types are already instantiated with unbounded values (*i.e.*, `List`) before the method defined as the entry point (*i.e.*, Line 3) starts executing. This is equivalent to declaring such values as *symbolic* in symbolic execution [22, 23, 35]. PROPCOV adds all custom types passed as arguments (*e.g.*, `UUID`) to the set of instantiated types, and the custom types those types define (*e.g.*, `UUID`’s fields), recursively until reaching primitive types, or types defined outside the SUT (*e.g.*, libraries or the Java framework). For completeness, PROPCOV traverses the class hierarchy repeating the search for all fields declared in all parent classes and implemented interfaces.

3.3 Optimizations

In the extensive evaluation of PBT we describe in Section 4, we found common patterns that reduced the accuracy of the analyses PROPCOV uses. In this section, we describe optimizations that PROPCOV uses to reduce such inaccuracies.

3.3.1 External code. The static analysis stage considers all the code inside the packaged System Under Test (SUT), including library code that the SUT imports and uses. However, PROPCOV follows JACOCO’s design philosophy and excludes external libraries from coverage reports, as SUT developers typically do not write the libraries that the SUT uses and are not interested in the coverage of the libraries’ code. Our design decision is pragmatic and allows PROPCOV to scale to large projects with many dependencies. However, discarding external code means that PROPCOV cannot reason accurately about rare cases where a library can call back into the SUT, which leads to PROPCOV missing reachable code. In practice, we found such cases to be rare: Section 4.1.1 explains how our empirical evaluation using 25 projects measured only 3% missing reachable LOC.

3.3.2 invokedynamic and Java lambdas. The JVM supports dynamic language features via `invokedynamic` [62]. For instance, Java uses `invokedynamic` to implement lambdas and stream

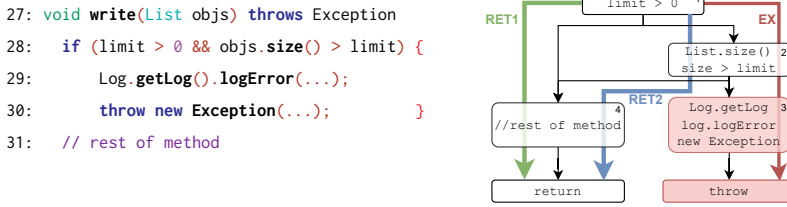


Fig. 5. Possible paths through code that throws exceptions.

operations. Java lambdas are typically defined inside the SUT but then passed to an external library. For instance, consider the following example using Java streams:

```

// Inside method m1, method m2 is part of the SUT
list.stream().forEach(i -> m2(i));

```

The main goal of the example above is to call method `m2` on each element in the provided list. Developers that write such code are unlikely to be interested in understanding how the inner workings of method `forEach` eventually leads to calling the provided lambda, and PropCov discards the implementation of `forEach` when performing its analysis. However, developers are likely to be interested in the much simpler fact that method `m1` calls method `m2`. Unfortunately, PropCov's handling of external code also discards such a fact.

The prevalence of Java lambdas requires PropCov to handle them carefully. Instead of just discarding all the information, PropCov leverages OPAL to recover the fact that the code in the SUT (e.g., `m1` in our example) calls the lambda also in the SUT, even if indirectly via third party code. OPAL rewrites invokedyynamic invocations to resemble static invocations that create a lambda object and then invoke it [57]. PropCov lists all paths that contain such rewritten lambda invocations, and simplifies them by adding edges from the original method (`m1`) to each edge departing the rewritten static invocation (`m2`). Even though it now looks like that the lambda can be called directly from the code that defines it, it captures the original intention of the developer and is much simpler than the whole path through external code unfamiliar (and even unknown) to the SUT developer.

3.3.3 Exceptions. In our experience, most exceptions thrown while performing PBT mean that the test found a bug and should terminate. For instance, consider the code on the left-hand side of Figure 5, adapted from `mph-table` [44]. In this example, throwing the exception on Line 30 means that this method was called on an invalid list of objects. Successful PBT executions *should never* cover Lines 29–30. However, PropCov considers such lines as reachable and always reports them as uncovered, which leads to a catch-22 report: these lines are not covered, and covering them requires a bug which then terminates PBT execution before reaching full coverage. Such nonsensical reports eroded our own trust in PropCov's results, and we designed an optimization to remove such code from reports.

The right-hand side of Figure 5 shows how PropCov skips code that throws exceptions. First, PropCov builds the control-flow graph (CFG) of basic blocks (BBs) for each method. Then, it lists all possible paths through the method, which end in either return or throw. Our example has 4 BBs and 3 paths: RET1, RET2, and EX. PropCov then lists all BBs only on the paths that end in throw: BB3 in our example. Finally, PropCov removes all methods/lines from the listed BBs from the results. As a result, PropCov reports method `write` as fully covered by a property test that

covers Line 28 (including all subexpressions in the `&&` expression) and Line 31. Of course, exception pruning is completely optional and can be disabled for projects in which it makes sense.

3.3.4 Iterable/Iterator. While evaluating PROPCOV, we noticed that projects with custom implementations of iterable and iterators caused RTA to lose precision when *any* iterator or iterable was used in the project when the analysis cannot track the origin of the collection (as shown in Figure 4b for method `serialize`). As such, PROPCOV’s iterable/iterator optimization simply removes all calls to iterable and iterator.

3.4 Combining analysis and coverage

PROPCOV combines the results of the static analysis with the actual coverage gathered by JACOCo when executing each property test. To provide recommendations to the developer about where to focus to improve a particular property test, PROPCOV computes a *score* per node in the call graph. The intuition behind the scoring is inspired from our own use of an early version of PROPCOV, when we noticed that the most interesting nodes have two properties: (1) they are close to the entry-point (*i.e.*, the property test itself), and (2) they can reach a large sub graph. Property 1 means that developers can connect the generator and the test readily to a particular line or method not being covered. Understanding how to connect the entry point to code far away from it (*i.e.*, needing a long sequence of many method calls to reach) is challenging, and we found that shallow uncovered nodes already lead to improvements in code coverage. Such a challenge can be best approached with automated techniques (*e.g.*, symbolic execution [22, 23, 35] or fuzzing [59, 72]), and we leave the integration of PROPCOV with such techniques as exciting future work, out of the scope of this paper. Property 2 maximizes the impact of reaching an uncovered line, as it “controls” a large portion of the code that was not reached before.

PROPCOV uses the following algorithm to compute the score of each node in the call graph, which can be a method or a line of code. Considering leaf nodes first, the score is either 0 for covered nodes, or 1 for nodes not covered. First, PROPCOV sets the score of all covered nodes to 0, and all uncovered nodes to 1. Then, PROPCOV traverses the graph in Depth First Search (DFS) order, starting from the deepest nodes and working its way back to the entry point. Leaf nodes retain their score. Branch nodes (parents) sum their original score with the score of each child — thus following Property 2 above — multiplied by 50% — thus implementing a decay function to follow Property 1 above. Once the algorithm finishes, PROPCOV lists all the paths in the graph from the entry-point to each uncovered leaf node, and computes the score of such path by simply summing the score of all methods on that path. Finally, PROPCOV suggests up to N paths to the developer. We found that N=3 works well in practice and used it for our evaluation, which we describe in Section 4.

Finally, PROPCOV presents a report as a colored method call graph similar to Figure 4, and visually highlights the N paths suggested, so that the developer can then follow each path from the property test to an uncovered red node (not shown in Figure 4).

3.5 Limitations

The current implementation of PROPCOV works only for projects that use Maven [17] as the build system. This is an engineering limitation, and we leave adding other build systems, such as Gradle [38], as future work.

Due to the pruning approach described in Section 3.3.1, PROPCOV misses method calls from overridden methods in parent classes outside of the SUT. For instance, the program below:

```
class Outside { void a(); void b() { a(); ... } } // Outside the SUT
class Inside extends Outside { void a() { ... } } // Inside the SUT
```

Class `Inside`, part of the SUT, extends class `Outside`, outside of the SUT. Both classes implement method `a`, called by method `Outside.b`. In this case, a property test that calls `Outside.b` can reach method `Inside.a`. PROPcov removes all code outside the SUT (Section 3.3.1), which has the unfortunate consequence of missing this case and (incorrectly) considering method `Inside.a` as unreachable. As such, PROPcov is not suitable for projects that depend heavily on this code pattern, or on external libraries calling back into the SUT.

Some PBT frameworks, such as `junit-quickcheck`, allow developers to annotate methods that should run before/after each property test/class. Although not a fundamental limitation, PROPcov's prototype does not support such annotations and misses all fields initialized this way, resulting in analyses RTA and k-CFA considering such fields as `null` and dropping all method calls that use them as the receiver object. We leave extending PROPcov to support such annotations as future work.

Some property tests only exercise other parts of the tests and no portion of the SUT. For instance, project `wires` [26] uses a property test to ensure that a custom generator outputs diverse values. PROPcov does not consider the coverage of tests, and returns a trivially false result in such cases (e.g., no code found).

4 Experimental Evaluation

In this section we measure the accuracy of PROPcov in terms of **Missed Reachable (MR)** code — identified as *not reachable* but that is actually reachable — and **Unfeasible Reachable (UR)** code — identified as *reachable* that is impossible to reach. We answer the following Research Questions (RQs), grouped into: measuring PROPcov's accuracy (RQs 1–3), and using PROPcov to conduct a novel study on the use of PBT (RQs 4–6):

RQ1 What is the accuracy of PROPcov in comparison to JACoCo in terms of: Missed Reachable (MR) rate, and Unfeasible Reachable (UR) rate?

RQ2 What is the trade-off in graph size versus accuracy between each type of analysis that PROPcov supports?

RQ3 How do PROPcov's optimizations impact the results?

RQ4 What is the existing coverage of PBT?

RQ5 What is the trade-off between increased coverage and test duration from simply increasing the number of trials in PBT?

RQ6 What is the increase in PBT quality (coverage and new bugs) when using PROPcov?

We found existing Java projects that use PBT on Github, searching for repositories that include Java sources that match the string `junit.quickcheck`. We found a total of 348 projects. The vast majority of these projects, 219, contain code used by students in academic classes on testing, which we discarded as the scope of this paper is not on how developers learn to use PBT. We also discarded 23 projects that include the source or modify `junit-quickcheck` itself, or use parametric fuzzing via JQF [47, 58]. We further discarded 36 projects that are just forks of other projects already included. The current prototype of PROPcov requires Maven [17], so we discarded 30 that use other build systems (e.g., Gradle [38]). We discarded 7 that do not use `junit-quickcheck` and 9 that do not build, or have failing property tests. To show that PROPcov can be used with other PBT frameworks, we added support for `jqwik` [1] and included one project that uses it: `occurrent`. In total, we considered the 25 shown in Table 1.

We ran all experiments on a machine equipped with 4 sockets, each holding an Intel(R) Xeon(R) Gold 5318H CPU (18 physical cores) and 194GB of RAM, running Ubuntu 22.04.3 LTS. We limited each experiment to a single socket.

Table 1. Projects used in the experimental evaluation. This table lists the 25 projects, totaling 293 property tests, and describes each project in terms of: ★ stars, 🍴 forks, 👤 contributors, 📄 number of commits, and Lines Of Code for the project. We analyze unit and property tests separately, showing, for each, the: # number of tests, Lines Of Code for the tests, and Time To Complete running all tests (averaged over 10 runs). Finally, we report how long PROPCov took to analyze all properties per project.

Project	Description	★	🍴	👤	📄	LOC	Unit Tests			Property Tests			PROPCov
							#	LOC	TTC (s)	#	LOC	TTC (s)	
3d-bin	Geometric computation	392	108	13	1018	19110	139	3753	146.45	3	115	22.39	238.69
beam-zstd	Compression	0	0	1	9	296	3	115	14.75	1	26	14.87	56.16
browserstack	Website testing	13	50	25	387	5049	10	427	41.61	1	16	41.35	90.15
capsyl	Code visualization	3	0	1	133	1778	65	826	12.96	1	32	11.7	36.43
convex	Internet of value	83	27	19	4106	47523	748	11182	54.88	3	660	34.25	1568.59
dyn-shield	Reinforcement learning	5	1	1	4	775	1	41	15.71	1	56	16.28	48.02
funcj	Functional data structures	99	10	4	739	30053	135	2256	34.81	55	937	27.01	1841.55
HprofCrawler	Heap dump tool	2	0	1	46	3412	4	38	11.49	11	123	13.27	329.16
im-collections	Immutable collections	0	0	2	44	1808	10	126	9.7	38	450	14.9	582.12
ipresource	IP resources	15	13	13	319	4388	240	2235	13.71	1	33	13.71	34.09
JESA	Event sourced aggregates	9	4	3	39	691	40	412	5.68	1	17	5.66	24.05
jflex	Lexer	574	111	19	2109	613063	85	1244	54.48	45	537	43.16	1689.8
jLens	Lenses for Java fields	0	0	1	3	298	2	23	9.23	9	122	11.25	133.59
libosu	Game API	1	1	1	63	2242	32	731	36.19	4	116	11.33	79.43
logish	Logical programming	0	0	3	50	4057	52	641	11.41	2	42	12.11	39.2
manu	Time series storage	3	0	1	140	5862	233	2793	24.03	13	390	24.17	344.95
mph-table	Hash table	95	19	7	55	5447	44	10977	13.53	8	50	11.85	202.35
NexappCore	REST API collection	3	3	5	95	1495	115	1283	10.56	1	128	10.53	28.95
occurrent	Event Sourcing Library	149	17	8	1403	26181	762	882	184.03	1	33	8.04	30.74
paillier4j	Cryptography	3	0	2	19	423	2	30	7.45	6	59	381.87	362.53
rdapd	Network API	8	3	8	586	7694	100	2297	29.07	1	37	28.83	53.54
rpki-commons	Resource certification	13	10	15	1269	20652	620	8216	33.82	6	335	36.43	338.87
sinch-sms	SMS/MMS API	3	5	12	542	9478	185	5471	26.59	20	499	25.77	624.53
stoa	Array utilities	0	0	2	7	6331	20	1116	28.32	38	933	30.72	1074.35
wires	Circuit simulator	1	1	1	345	4969	107	1742	16.35	23	404	59.14	536.65

Unless otherwise stated, the results use PROPCov’s RTA support and all optimization techniques enabled (Section 3.3). We use RTA as default because, as this section shows, RTA provides a good compromise between accuracy and time.

In our experimental evaluation, Table 1 shows that PROPCov’s longest execution time is approximately 30 minutes for large projects (convex, funcj, jflex), which aligns with how we envision PROPCov being used: during “nightly” builds with fresh results ready for developers to review in the next morning.

4.1 PROPCov’s Accuracy

4.1.1 PROPCov’s MR. The first experiment measures how many lines PROPCov considered not reachable that are indeed reachable: Missed Reachable (MR). The **ground-truth** is the coverage reports produced by JACoCo. We consider as MR lines that JACoCo observes to be reached during PBT execution but PROPCov does not include in its reports. Such lines are cases where PROPCov’s analysis under-approximates the SUT. Table 2 shows the result as a percentage of lines reached over lines reachable: PROPCov’s results are relative to the lines it deems reachable for each property (i.e., a different denominator per property), and JACoCo’s results are relative to all lines in the project (excluding tests) as JACoCo considers the whole project reachable (i.e., all properties have the same denominator). Some projects in Table 2 have a higher MR for methods than lines (e.g., jLens) due to many small lambdas defined on the same line, idiomatic when using functional Java features such as streams.

Table 2. Missed Reachable (MR) and Unfeasible Reachable (UR) statistics for PROPcov and JACoCo. The Lines and Methods columns report average counts across all properties for each project, with relative percentage shown in parenthesis. The final row reports an average over all the properties and projects, rather than an average of the rows above. By definition, JACoCo’s MR is always zero and is therefore omitted. We approximate PROPcov’s UR by considering all unreached nodes in the identified properties as unfeasible. We note that JACoCo considers code not visible to PROPcov (e.g., static variable initializers), hence the small difference between PROPcov’s coverage plus MR when compared to JACoCo’s coverage.

Project	PROPcov						JACoCo					
	Coverage		MR		UR		Coverage		UR			
	Lines	Methods	Lines	Methods	Lines	Methods	Lines	Methods	Lines	Methods	Lines	Methods
3d-bin	317 (75%)	101 (81%)	1559 (12%)	380 (20%)	0 (0%)	0 (0%)	1876 (14%)	481 (25%)	11334 (87%)	1449 (77%)		
beam-zstd	24 (77%)	5 (100%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	24 (51%)	5 (50%)	16 (34%)	5 (50%)		
browserstack	25 (68%)	6 (86%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	25 (1%)	6 (1%)	2179 (98%)	433 (98%)		
capsyl	1 (100%)	1 (100%)	3 (1%)	1 (1%)	0 (0%)	0 (0%)	4 (1%)	2 (1%)	314 (99%)	165 (99%)		
convex	2141 (64%)	714 (61%)	3569 (14%)	954 (20%)	0 (0%)	0 (0%)	5721 (23%)	1669 (35%)	21285 (86%)	3805 (79%)		
dyn-shield	7 (2%)	2 (5%)	8 (2%)	3 (4%)	0 (0%)	0 (0%)	15 (4%)	5 (7%)	38 (9%)	24 (34%)		
funcj	210 (92%)	150 (74%)	60 (1%)	41 (1%)	3 (0%)	3 (0%)	270 (3%)	191 (5%)	8520 (97%)	4025 (96%)		
HprofCrawler	81 (82%)	28 (93%)	48 (3%)	6 (2%)	0 (0%)	0 (0%)	131 (9%)	34 (10%)	1434 (94%)	296 (91%)		
im-collections	449 (94%)	135 (87%)	55 (9%)	52 (24%)	21 (4%)	3 (1%)	509 (87%)	187 (86%)	107 (18%)	82 (38%)		
ipresource	71 (61%)	28 (85%)	92 (9%)	30 (11%)	0 (0%)	0 (0%)	163 (16%)	58 (21%)	833 (80%)	217 (77%)		
JESA	20 (91%)	7 (100%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	20 (22%)	7 (25%)	71 (76%)	21 (75%)		
jflex	354 (85%)	69 (95%)	66 (1%)	16 (2%)	0 (0%)	0 (0%)	420 (7%)	85 (12%)	5703 (93%)	634 (90%)		
jLens	45 (96%)	20 (71%)	3 (5%)	12 (32%)	0 (0%)	0 (0%)	48 (75%)	32 (86%)	17 (27%)	11 (30%)		
libosu	76 (95%)	31 (86%)	16 (1%)	6 (1%)	0 (0%)	0 (0%)	92 (5%)	37 (9%)	1702 (96%)	396 (93%)		
logish	86 (76%)	21 (31%)	28 (2%)	9 (2%)	0 (0%)	0 (0%)	114 (6%)	30 (5%)	1678 (92%)	551 (94%)		
manu	416 (88%)	69 (97%)	233 (14%)	30 (13%)	0 (0%)	0 (0%)	651 (38%)	99 (44%)	1237 (72%)	156 (69%)		
mph-table	78 (53%)	34 (85%)	11 (1%)	8 (2%)	0 (0%)	0 (0%)	89 (5%)	42 (9%)	1693 (92%)	413 (91%)		
NexappCore	0 (0%)	0 (0%)	8 (4%)	3 (3%)	7 (3%)	2 (2%)	8 (4%)	3 (3%)	199 (96%)	89 (97%)		
occurrent	17 (50%)	4 (100%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	17 (11%)	4 (6%)	116 (77%)	60 (94%)		
paillier4j	40 (93%)	24 (100%)	39 (37%)	6 (15%)	0 (0%)	0 (0%)	79 (76%)	30 (73%)	25 (24%)	11 (27%)		
rdapd	16 (89%)	5 (100%)	87 (4%)	12 (2%)	0 (0%)	0 (0%)	103 (5%)	17 (3%)	1935 (95%)	659 (97%)		
rpki-commons	1314 (75%)	373 (95%)	83 (2%)	37 (2%)	0 (0%)	0 (0%)	1397 (26%)	410 (27%)	3640 (67%)	1123 (73%)		
sinch-sms	451 (90%)	176 (99%)	159 (2%)	85 (4%)	0 (0%)	0 (0%)	611 (8%)	261 (11%)	6850 (93%)	2118 (92%)		
stoa	1033 (77%)	236 (82%)	125 (6%)	65 (14%)	15 (1%)	2 (0%)	1175 (60%)	303 (63%)	600 (31%)	192 (40%)		
wires	31 (72%)	11 (55%)	12 (1%)	7 (2%)	4 (0%)	1 (0%)	43 (5%)	18 (4%)	901 (94%)	383 (94%)		
	292 (72%)	90 (74%)	125 (3%)	49 (4%)	2 (0%)	0.44 (0%)	544 (16%)	161 (20%)	2897 (86%)	693 (85%)		

4.1.2 PROPcov’s UR. In this experiment, we measure how PROPcov over-approximates the SUT by counting how many properties have reachable (but not reached) nodes that are in fact unfeasible: Unfeasible Reachable (UR). We ran this experiment using 1-CFA instead of RTA to ensure maximum accuracy. Unfortunately, this experiment does not have a factual source of **ground-truth**. Instead, we manually analyzed all the reports with unreached nodes that PROPcov produced, for a total of 165 properties. For each unreached node, we executed the SUT inside the debugger to understand how the node could be reached. Table 2 shows the results.

For properties with UR (i.e., nodes we classify as not feasible), we consider all unreached nodes as UR, which is conservative against PROPcov because some unreached nodes are legitimate, but provides a conservative worst-case number without expert knowledge of each individual project and property to manually analyze (hundreds of) nodes.

We can see that most projects (20) have zero UR. Unreached nodes in these projects *can* be reached, but the current test and/or generator are not able to do so. In total, we found 19 properties with UR (6.4%). 1-CFA struggled with 1 property on *funcj* – it connected an application of a lambda to an (incorrect) toString method – and *stoa* – 8 properties rely on Java reflection, which is not supported. PROPcov’s limitations lead to UR on properties that target only the generator – 6 on *wires* – and properties that rely on fields inherited from classes outside the SUT – 3 on *im-collections*, and 1 on *nexappcore*.

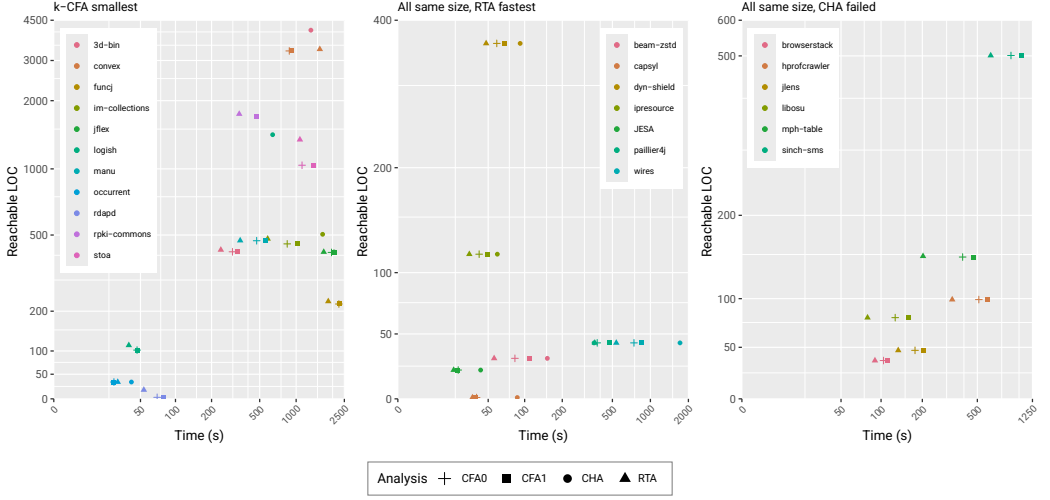


Fig. 6. Size of generated graph versus time taken to generate the graph per analysis, clustered according to size and speed of each analysis.

4.1.3 JACoCo's UR. We used PROPCov's results to measure how much unfeasible code JACoCo's results show as reachable. We use as **ground-truth** the results of PROPCov that we manually validated (Section 4.1.2) and count all the lines and methods that JACoCo reports as not covered but that PROPCov reports as unreachable from the original property. Table 2 shows the results.

Answer to RQ1: Our empirical evidence, from the 25 projects in our corpus, suggests that: (1) PROPCov's MR is low, PROPCov misses only 3% lines of code that are actually reachable (4% of all the methods); (2) PROPCov's UR is low, we only found UR on 19 properties (6.4% rate) over 5 projects, on average affecting 2 lines over 0.44 methods per project; and (3) JACoCo's UR is high, an average of 86% lines over 85% methods are UR. PROPCov's UR further break down to 9 properties (3%) due to limitations of the underlying analysis, and 10 properties (3.4%) due to limitations of PROPCov itself. In our evaluation, JACoCo's reports cause developers to consider, on average, 2897 lines over 693 methods that cannot be possibly reached from any property, which is a strong suggestion that JACoCo alone is not suitable for PBT, as we discuss in Section 2.1.

4.1.4 PROPCov's different analyses. PROPCov uses OPAL's analyses to compute the reachability from each property test. In this section, we show the impact of considering each analysis — CHA, RTA, 0-CFA, and 1-CFA. We measure the impact of each analysis in terms of how much more code it includes, and time to complete. We manually clustered similar results, as shown in Figure 6.

1. k-CFA smallest (11 projects). k-CFA removes more unfeasible nodes, resulting in a smaller graph produced faster than all other analyses. Still, we note that RTA is always close to k-CFA and often faster (9 projects).

2. All have same size and RTA is fastest (7 projects). The underlying projects use virtual methods with only one concrete implementation, and the small difference in time is due to inefficiencies in the implementation of each analysis.

3. All have same size, RTA is fastest, and CHA fails (6 projects). CHA's straightforward approach can result in a compounding increase in the number of nodes considered (adding more nodes makes even more nodes reachable, and so on), which causes CHA to fail with either a timeout

or an out-of-memory error. In all such cases, RTA and k-CFA tie for graph size, with RTA being faster.

4. k-CFA and RTA fail (1 project). Limitations of PROPcov (Section 3.5) cause k-CFA and RTA to not find any SUT code. CHA simply lists *all* possible concrete implementations, and is thus able to find *some* SUT code. The only project in this cluster is *nexappcore* (omitted from Figure 6), for which CHA finds around 15 LOC in 49 seconds.

We also computed the MR rate for each analysis using the same methodology as Section 4.1.1. 17 projects have the same MR rate for RTA and both k-CFA, and 5 projects have a negligible difference (lower than 0.05%). Surprisingly, k-CFA did not minimize the MR rate for any remaining project, which means that the analysis, or how PROPcov configures it (Section 3.2.2), under-approximates the SUT and discards nodes that can be reached. For the remaining 3 projects, k-CFA has the higher MR rate than RTA: *im-collections* (13% line, 26% method), *rdapd* (5% line, 2% method) and *stoa* (16% line, 25% method).

Answer to RQ2: RTA is always the fastest in 22 projects: for 13 projects, RTA and k-CFA result in the same smallest graph size; and, for 9 projects, k-CFA generates smaller graphs but RTA is close. CHA never produces small or fast results. RTA also always generates graphs with the lowest MR. Our results show that RTA is the preferred analysis as it provides the fastest best results for most projects.

4.1.5 Impact of PROPcov's optimizations. Section 3.3 describes PROPcov's optimizations for common language features: Java lambdas, iterators/iterable, and exceptions. We measured the impact of these optimizations on all projects by rerunning PROPcov with different configurations: no optimizations ($\emptyset$), Java lambda (λ), iterators/iterable (I) and exceptions (E) independently but together with λ , and all optimizations. Table 3 shows the results for RTA and 0-CFA. We omit the results for: projects with no changes; 1-CFA, which are similar to 0-CFA; and CHA, which are much worse than RTA.

Each optimization needs to reach code with their target feature, and no optimization triggered for 176 properties for RTA and 185 for k-CFA. Most optimizations have more impact on RTA than 0-CFA, possibly due to 0-CFA's higher accuracy [61]. Note that combining optimizations is not linear, as two optimizations can remove the same code (e.g., a custom iterator catching exceptions). Finally, we also computed the MR for each configuration using the same methodology as Section 4.1.1. The λ optimization included missing reached code, the I and E optimizations removed reached code on projects that use custom iterators/iterable or use exception handling for non-terminal errors, respectively. We can see that, in terms of callgraph reduction, 0-CFA benefits less from the pruning strategies as it has smaller callgraphs than RTA to start with. Still, the pruning strategies reduce the overall callgraph size for both analyses, with a small price in accuracy (i.e., increased MR).

Answer to RQ3: PROPcov's optimizations change the results of 117|108 (for RTA|k-CFA) properties by adding 440|327 feasible nodes reached through lambdas, removing 161|101 unfeasible nodes due to the analyses over-approximation of iterators/iterable, and ignoring exception handling nodes. On average PROPcov's optimizations find 33.9|25.2 more feasible nodes, and remove 12.4|7.8 unfeasible nodes. The optimizations have no benefit for 13 projects, because such projects do not use any target feature. The more accurate analyses k-CFA benefit less from the optimizations as they are more accurate to start with. The λ optimization always increases accuracy, reducing the MR by 20.0|14.8 LOC on average. The I and E optimizations drop uninteresting code that is reached, resulting in a modest MR increase of 5.3|3.2 LOC and 1.91|1.55 LOC, respectively; and 7.1|4.6 LOC when combined.

Table 3. Nodes added by λ functions and removed by the **Iterator** and **Exception** pruning strategies from the original graph $\emptyset$. This table reports how many properties do not change (**0**), all properties of that project (**n**), and the average: node change (top number) and MR line change (bottom number) per project. We omit: CHA, as it is very inaccurate, 1-CFA, as it is similar to 0-CFA, 12 projects (53 properties) without change, and the change in MR for methods. Column (λ) compares to ($\emptyset$); columns (λ, I), (λ, E), and (λ, I, E) compare to column (λ). Cells with $-$ denote no change.

Project	0/n	RTA					0/n	0-CFA				
		$\emptyset$	λ	λ, I	λ, E	λ, I, E		$\emptyset$	λ	λ, I	λ, E	λ, I, E
convex	0/3	1228 MR 3517	+10 MR -34	-38 MR +49	-24 MR +43	-62 MR +88	0/3	973 MR 3545	+1 MR +2	-15 MR +8	-23 MR +28	-38 MR +19
dyn-shield	0/1	11 MR 8	+32 MR 0	—	—	—	0/1	10 MR 8	+33 MR 0	—	—	—
funcj	23/55	110 MR 129	+94 MR -69	—	—	—	32/55	99 MR 129	+88 MR -69	—	—	—
hprofcrawler	11/11	26 MR 48	+4 MR 0	—	—	—	11/11	26 MR 48	+4 MR 0	—	—	—
im-collections	26/38	158 MR 24	+22 MR -10	-25 MR +41	— MR +41	-25	29/38	142 MR 71	+27 MR -31	-23 MR +40	—	-23 MR +40
jflex	30/45	79 MR 36	—	-6 MR +30	— MR +30	-6	23/45	38 MR 226	—	-6 MR +17	—	-6 MR +17
jlens	2/9	17 MR 5	+11 MR -2	—	—	—	2/9	17 MR 5	+7 MR -2	—	—	—
libosu	2/4	23 MR 50	+13 MR -34	—	—	—	2/4	13 MR 55	+18 MR -39	—	—	—
logish	0/2	68 MR 4	+4 MR -1	-7 MR +25	+1 MR 0	-9 MR +25	0/2	17 MR 41	+4 MR -5	—	—	—
manu	9/13	102 MR 44	—	-15 MR +142	-12 MR +46	-31 MR +189	9/13	60 MR 45	—	-6 MR +65	-2 MR +2	-8 MR +67
rdapd	0/1	1 MR 102	+4 MR -15	—	—	—	1/1	1 MR 102	—	—	—	—
sinch-sms	12/20	197 MR 135	—	-16 MR +24	-7 MR 0	-19 MR +24	14/20	197 MR 135	—	-16 MR +24	-7 MR 0	-19 MR +24
stoa	8/38	52 MR 878	+246 MR -779	-9 MR +26	—	-9 MR +26	9/38	43 MR 908	+145 MR -504	-5 MR +18	-2 MR +2	-7 MR +20

4.2 Study of existing PBT

4.2.1 Coverage of existing PBT. PROPcov provides a novel insight into the coverage of existing PBT. We start by comparing its coverage results per property against JACoCo by running each tool on each property. Table 2 shows the results.

Answer to RQ4: PROPcov reports that, for existing PBT, 72% lines and 74% methods are covered. JACoCo reports 16% lines and 20% methods. PROPcov’s results show that there is room to improve PBT and that JACoCo alone is not suitable for PBT as it reports inaccurate coverage.

4.2.2 Impact of the number of trials. Given the room for improvement described in Section 4.2.1, developers can simply increase the number of trials in PBT. We used PROPcov to gain novel insight into how trials impact PBT coverage by varying the number of trials, and measuring the coverage and time per property. We ran each property for each number of trials 10 times, and clustered the results according to maximum line coverage. We note that Arcuri and Briand [18] recommend at least $n=10$ for a “large number of artifacts (...) but there are constraints in terms of execution time” with the specific example of (unit) testing. Our setting (293 properties $\times$ 5 trial counts) satisfies this “large number of artifacts” criterion. Table 4 shows the results, as a descriptive cluster-level summary of the observed coverage.

We found 74 tests reaching full coverage: 51 with 1 trial, 22 with 10, and 1 with 100. Inspecting the tests with 1 trial reveals straight line code with three possible explanations. **First**, these tests are very simple and exercise the SUT in a trivial way. For instance, calling a SUT method that returns an input lambda wrapped in another lambda, which then the test uses more sophisticated logic

Table 4. Impact of varying the number of trials per line coverage and time, clustered per number of trials to reach maximum coverage. This table does not include 73 tests because they require more than 60 minutes to run 10 or more trials, or because they fail at low numbers of trials (e.g., ensuring a generator has at least a 1% chance of generating a particular value).

Cluster	n	Coverage per trials					Time (s) per test				
		1	10	100	500	1000	1	10	100	500	1000
100% at 1	51	100%	100%	100%	100%	100%	13.41	13.42	13.53	13.72	14.16
Max at 1	54	80%	79%	78%	80%	80%	15.22	15.32	15.55	18.18	30.74
100% at 10	22	54%	100%	100%	100%	100%	10.75	10.7	10.94	11.41	12.45
Max at 10	46	50%	82%	80%	82%	82%	13.3	13.52	13.72	20.2	60.08
100% at 100	1	44%	89%	100%	100%	100%	20.34	20.85	20.83	20.92	21.67
Max at 100	34	42%	68%	84%	73%	73%	14.34	14.49	15.04	21.2	50.54
Max at 500	10	49%	68%	68%	79%	79%	8.7	9	10.06	23.51	88.88
Max at 1000	2	68%	71%	68%	68%	80%	13.64	13.39	13.8	14.89	15.84

to ensure the wrapped lambda behaves as expected. **Second**, the tests sample a large input space. For instance, a SUT method that turns longs into byte arrays has straight line code, but it makes sense to write a property test to sample the space of inputs and ensure that it behaves as expected. Finally, **third**, the tests involve either loops executed at least once or Java streams. In both cases, at least one iteration results in full line coverage. PROPcov can be adapted to use better notions of coverage, such as static path coverage from JACoCo (i.e., only 2 static paths per branch/loop: taken or not taken), and dynamic path coverage from fuzzers (e.g., AFL considers the number of loop iterations as different paths: 1, 2, 4, 8, and so on [72]).

For all tests, we did not see an increase in time taken to run each property until 100 trials. Increasing the trials to 500 and 1000 resulted in reaching the maximum coverage for 10 and 2 tests, respectively, while increasing the duration for 26 and 28 tests, respectively (not shown in Table 4).

Answer to RQ5: Most tests (173) reached their 100% or maximum coverage with 10 trials. 100 trials further increased the coverage on only 35 tests, without increasing the time required to run the test. 500 or more trials further increased the coverage for only 12 tests, with significant slowdowns for 26 tests with 500 trials, and 28 tests for 1000 trials. Increasing the number of trials alone typically does not lead to increased coverage. Our results suggest that 10 is a good default number of trials for most properties (instead of 100 for `junit-quickcheck`), and that developers should use PROPcov to understand if each particular property should use more trials.

4.2.3 Using PROPcov with existing PBT. When conducting the manual analysis in Section 4.1.2, we also attempted to improve the quality of existing PBT by changing the generator or the test to reach previously unreachable code. As non-specialists in any of the projects, we used the following protocol. We divided the work amongst authors and defined maximum effort in time per project as follows: projects with 5 properties or less receive half a day of attention (e.g., 4 hours), between 5 and 10 properties receive one full day, and more than 10 properties receive two full days. While exploring each project, we only considered properties with any uncovered nodes and followed PROPcov’s heuristic (Section 3.4), running each property in the debugger to understand how to reach (or not) any of the highest ranking uncovered nodes. Our original efforts considered the highest 5 ranking paths, but we noticed within the first day that it was too much information to present at once, and we configured PROPcov to show only 3 paths from then onwards. Junior authors performed the exploration described above, and then confirmed their findings with senior authors during daily 1h meetings. Turning our findings into pull requests and bug reports took more effort to polish the examples and code submitted to the standards of external developers,

Table 5. Projects improved by following PROPcov’s suggestions. This table reports coverage as measured by PROPcov as the reduction in uncovered methods/lines.

Project	# Props	Methods				LOCs			
		Total	Before	After	Improv	Total	Before	After	Improv
funcj	1	16	15	16	100%	17	16	17	100%
jflex	7	46	35	46	100%	363	193	297	61.18%
manu	3	40	32	38	75%	261	184	210	33.77%
mph-table	1	27	21	24	50%	120	52	83	45.59%
sinch-sms	4	102	98	102	100%	297	272	284	48%
stoa	10	245	178	193	22.39%	1085	696	776	20.57%
wires	16	20	13	17	57.14%	43	29	37	57.14%
Total	42			Average:	72.08%			Average:	52.32%

depending on the complexity and size of each project, but never more than 2 days. Table 5 shows the results.

In total, we improved the coverage of 42 tests over 7 projects, and found 5 previously unknown bugs.

We opened 9 pull requests and 2 issues. At the time of writing, 5 pull requests were merged [2–4, 6, 14], and 2 bugs [5, 15] were confirmed and fixed by the developers. The remaining 4 pull requests remain open [7, 10–12]. For the rest of the projects we did not improve, given our lack of familiarity, it was unclear if we could reach lines deep in the graph via either: a particular value from the generator to satisfy a check, or a change to the test itself.

PROPcov shows that high coverage in PBT does not necessarily mean absence of bugs. For instance, funcj started with 92% method and 74% line coverage. Despite such values, improving PBT still revealed a bug, which shows PBT’s ability to find subtle bugs in corner-cases even for a modest improvement in overall coverage. Using PROPcov, we reduced uncovered methods/lines by an average of 72.08%/52.32%, reaching 100% method coverage for 12 properties over 3 projects, and 100% line coverage for 1 test. We found the following bugs:

convex An existing test did not cover the data-type blob. We improved the generator by adding the ability to generate blob instances of varying size and with random contents, and we found an error for blobs larger than 4KB. This issue was reported, acknowledged, and fixed.

funcj The optional Either must short-circuit: `Either.left(x).left(y)` must result in `Either.left(x)`. We added a case to the test that should short-circuit as described, and found that it resulted in `Either.left(y)`. The revealing input requires a particular deletion on empty integer maps, clearly a corner-case that the developers missed. This issue was reported, acknowledged, and fixed.

logish We found null checks that always failed when inserting/removing items from a hash map, and improved the tests to also insert/remove null items. We found two bugs, one for insertion and one for deletion. These issues are still pending.

manu On a test to roundtrip encoding and decoding IDs, the code to encode small and medium numbers was not covered. We changed the test to generate diverse number sizes, and found one bug which attempts to encode medium-sized IDs with not enough bits, resulting in a different ID being decoded. This issue is still pending.

Answer to RQ6: We used PROPcov’s guidance to improve 42 tests over 7 projects, reducing uncovered methods/lines by 72.08%/52.32% and finding 5 previously unknown bugs in the process. The original developers confirmed our findings by merging 5 pull-requests and fixing 2 bugs. We

also reached 100% method coverage for 12 properties over 3 projects, and 100% line coverage on 1 property.

5 Related Work

Based on a user-study of developers using PBT [36], Tyche [37] provides different visualizations to help developers understand PBT results, including how coverage increases with each fresh trial/input. Tyche considers 100% coverage whatever the PBT test reaches during a run, and plots how that coverage increases with more trials. Wauters and De Roover [68] report a similar need in open-source ML projects: Developers adopt Hypothesis expecting broader test coverage, but lack the means to assess what their properties and generators actually exercise. PROPcov provides observed/possible coverage per test, which allow developers to gauge if a particular test already covers the intended functionality, and, if not, PROPcov provides actionable suggestions on how to improve such a test. PROPcov uses OPAL to approximate PBT coverage.

Parameterized Unit Testing (PUT) [66] also uses tests with arguments (parameters) but for a different purpose. Instead of using a generator to sample the same test with different random values, PUT relies on symbolic execution to reason about all possible inputs by making arguments symbolic and collecting different Path Conditions (PC). Then, PUT uses a solver for each different PC to generate a concrete test (*i.e.*, concrete solution to the test's arguments) representative of that PC. PUT can provide a precise notion of path-based test completeness, but suffers from the inherent limitations of symbolic execution (*e.g.*, path explosion, lack of support for external code) [24, 47, 48], which does not affect PBT (and PROPcov by extension).

A complementary body of work focuses on comparing test suites (without 100% coverage), finding branch coverage to be hard to improve upon [34], and that coverage of even unchanged code shifts as software evolves [32, 41, 51]. When measuring per-test statement coverage as unit tests evolve into property-based tests, Wauters *et al.* [69], lacking a per-test notion of reachable code, approximate the code reachable from each test by normalizing executed statements by those in the files each test executes. PROPcov provides exactly such a per-test denominator as a ceiling of theoretical maximum reachability, used when computing coverage per property test, enabling principled coverage studies of PBT suites analogous to those above.

PBT simply re-executes the same test a fixed number of times, each with different input. Random-testing, fuzz testing, or *fuzzing* [52], also re-executes the same code repeatedly with different random inputs. However, fuzzing is typically guided by code coverage, and employs many techniques to maximize coverage by modifying the random input in particular ways [33, 45, 72]. Fuzzing campaigns are much longer in length than PBT, ranging from hours to days [46], and use unstructured input, unlike PBT. Similarly to PROPcov, FuzzInspector [25] approximates the maximum possible coverage via the whole-program call-graph. FuzzInspector targets fuzzing harnesses, and PROPcov targets PBT. FuzzInspector considers the whole application instead of a single test, resulting in a coarse approximation of the program behavior that suffers from over- and under-counting [49] as seen in FuzzInspector's database entry for fuzzing `jflex` [8], which reports 38.58% method coverage with 61% MR (156/254). PROPcov provides high-quality feedback that guides developers towards code locations that kept PBT from making progress, and reports 95% method coverage with 2% MR for `jflex`.

Another main difference between fuzzing and PBT is that fuzzing does not guarantee a valid input for each execution, which is somewhat useful as it allows fuzzers to detect bugs when processing invalid input, but severely limits the applicability of fuzzing on programs with complex input because all inputs can be expected to be invalid. Some fuzzers get around this limitation by using a sophisticated generator of valid inputs. For instance, Csmith [71] generates random C programs guaranteed to be valid (*i.e.*, syntactically correct and without undefined behavior) to

test C compilers. More generally, *parametric fuzzing* [47, 58, 59] uses generators to obtain valid input that is then passed to the Software Under Test (SUT). Each random bit provided is now a parameter that the generator uses, such that different unstructured bits control different parts of the structured output. Given the similarity between parametric fuzzing and PBT, PROPCOV can use a similar analysis and heuristic on parametric fuzzing targets, thus providing a list of “candidate” high-value unreachable nodes for the fuzzer to focus on.

Some properties may keep program state between runs or rely on non-determinism (e.g., multi-threading), which PROPCOV leaves out of scope. Existing techniques for Checkpoint/Rollback [19, 21] and Record/Replay [20, 55, 63, 64] can be combined with PROPCOV to enable support for such properties.

PROPCOV focuses on PBT, with test suites that do not target the whole program. Other specialized testing methodologies also do not target the whole program (e.g., testing software update timing [39, 60]) and can benefit from PROPCOV’s approach to measuring coverage.

PROPCOV relies on call-graphs to perform forward reachability analysis from a particular method (the property test) to approximate the maximum reachability. *Program slicing* [65, 70] solves a different problem: what statements may be affected by a criterion (e.g., what statements may influence the value of a set of variables). PROPCOV’s architecture is modular (Figure 3), which can allow a future version to replace OPAL with a program slicer provided with appropriate criteria per property (e.g., an assertion).

6 Conclusion

This paper presents a novel mechanism for measuring the coverage of PBT in a meaningful manner on a per-test basis, especially suited for imperative languages such as Java, that combines a static analysis, to compute an approximation of the maximum theoretical coverage of each property test, with coverage-based dynamic analysis. Based on such a novel mechanism, this paper presents the design and implementation of PROPCOV, an extensible tool to guide developers using PBT, together with an extensive evaluation using 293 properties over 25 Java projects using PBT frameworks junit-quickcheck and jwik. PROPCOV results are accurate — only 6.4% of all properties contain unfeasible code, and PROPCOV only misses 3% of reachable code. PROPCOV uses a novel heuristic that scores program paths according to how close they were to be reached during execution, and suggests program points to improve. Following PROPCOV’s recommendations, we improved the coverage of 42 properties and found 5 bugs, all on projects unfamiliar to us. At the time of writing, 2 bugs were confirmed and fixed by the developers, and we submitted 9 pull requests, 5 merged by the developers (other bugs/PRs are still pending).

7 Data Availability

We have released both the source code of PROPCOV’s prototype [28] and a replication artifact [27], as a VM image for x86_64, that allows future researchers to reproduce the results in this paper, and to build on PROPCOV. The artifact contains the source of PROPCOV’s current prototype, scripts to run all experiments described in the paper, and the data we used.

Acknowledgments

This work was funded in part by the National Science Foundation (NSF) grant CCF-2227183. We would like to thank: John Fike, for IT support during development and testing; Will Cygan, for implementing an early prototype of PROPCOV; Alekh Mekka, Zara Farin, and Maazur Rahman for their efforts in implementing PROPCOV’s prototype; and our anonymous reviewers for their useful feedback.

References

- [1] 2017. jqwik: Property-Based Testing in Java. <https://jqwik.net/>. Accessed: 2026-07-13.
- [2] 2022. Improve coverage by varying the list size. <https://github.com/indeedeng/mpm-table/pull/9>.
- [3] 2022. Improve coverage of property tests by including various sizes of blob types. <https://github.com/Convex-Dev/convex/pull/356>.
- [4] 2022. Improved property tests to cover byte, char, short, float, and long. <https://github.com/adarshsoodan/stoa/pull/2>.
- [5] 2022. Large blobs (>4KB) fail the round trip assertion property tests. <https://github.com/Convex-Dev/convex/issues/357>.
- [6] 2022. Property test improvement. <https://github.com/jflex-de/jflex/pull/953>.
- [7] 2024. Better coverage for property tests. <https://github.com/vandekerker/wires/pull/2>.
- [8] 2024. FuzzInspector results: jflex. https://storage.googleapis.com/oss-fuzz-introspector/jflex/inspector-report/20251123/fuzz_report.html#functions_cov_hit_0. Accessed: 2026-07-20.
- [9] 2024. Haskell Discourse — The sad state of property-based testing libraries. <https://discourse.haskell.org/t/the-sad-state-of-property-based-testing-libraries/9880>. Accessed: 2026-07-20.
- [10] 2024. Improved coverage of property tests. <https://github.com/sinch/sinch-java-sms/pull/46>.
- [11] 2024. Improved property test coverage and found bugs. <https://github.com/cldellow/manu/pull/20>.
- [12] 2024. Improved property tests with null examples that reveals bugs. <https://github.com/dragan-ivanovic/logish/pull/3>.
- [13] 2024. SuperCollider: Property-Based Testing — Call for Opinions, Comments. <https://scsynth.org/t/property-based-testing-call-for-opinions-comments/9087>. Accessed: 2026-07-13.
- [14] 2024. Test coverage for fail, fail case. <https://github.com/typemeta/funcj/pull/35>.
- [15] 2025. Possible bug with Either.Left short-circuiting. <https://github.com/typemeta/funcj/issues/40>.
- [16] Paul Ammann and Jeff Offutt. 2008. *Introduction to Software Testing* (1 ed.). Cambridge University Press, USA. <https://doi.org/10.1017/CBO9780511809163>
- [17] Apache Foundation. 2002. Apache Maven is a software project management and comprehension tool. <https://maven.apache.org/>. Accessed: 2026-03-13.
- [18] Andrea Arcuri and Lionel Briand. 2014. A Hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Softw. Test. Verif. Reliab.* 24, 3 (May 2014), 219–250. <https://doi.org/10.1002/stvr.1486>
- [19] Jonathan Bell and Luís Pina. 2018. CROCHET: Checkpoint and Rollback via Lightweight Heap Traversal on Stock JVMs. In *32nd European Conference on Object-Oriented Programming (ECOOP 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 109)*, Todd Millstein (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:31. <https://doi.org/10.4230/LIPIcs.ECOOP.2018.17>
- [20] Michael D. Bond, Milind Kulkarni, Man Cao, Meisam Fathi Salmi, and Jipeng Huang. 2015. Efficient Deterministic Replay of Multithreaded Executions in a Managed Language Virtual Machine. In *Proceedings of the Principles and Practices of Programming on The Java Platform (Melbourne, FL, USA) (PPPJ ’15)*. Association for Computing Machinery, New York, NY, USA, 90–101. <https://doi.org/10.1145/2807426.2807434>
- [21] Rodrigo Bruno and Paulo Ferreira. 2016. ALMA: GC-assisted JVM Live Migration for Java Server Applications. In *Proceedings of the 17th International Middleware Conference (Trento, Italy) (Middleware ’16)*. Association for Computing Machinery, New York, NY, USA, Article 5, 14 pages. <https://doi.org/10.1145/2988336.2988341>
- [22] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (San Diego, California) (OSDI’08)*. USENIX Association, USA, 209–224.
- [23] Cristian Cadar, Vijay Ganesh, Peter M. Pawlowski, David L. Dill, and Dawson R. Engler. 2008. EXE: Automatically Generating Inputs of Death. *ACM Trans. Inf. Syst. Secur.* 12, 2, Article 10 (Dec. 2008), 38 pages. <https://doi.org/10.1145/1455518.1455522>
- [24] Cristian Cadar and Koushik Sen. 2013. Symbolic execution for software testing: three decades later. *Commun. ACM* 56, 2 (Feb. 2013), 82–90. <https://doi.org/10.1145/2408776.2408795>
- [25] Oliver Chang, Navid Emamdoust, Adam Korczynski, , and David Korczynski. 2016. Introducing fuzz introspector, an openssf tool to improve fuzzing coverage. <https://openssf.org/blog/2022/06/09/introducing-fuzz-introspector-an-openssf-tool-to-improve-fuzzing-coverage/>. Accessed: 2026-07-05.
- [26] Laurent Claisse. 2017. A Java port of the “Simulator for digital circuits” from SICP (Structure and Interpretation of Computer Programs). <https://github.com/vandekerker/wires>. Accessed: 2026-07-13.
- [27] Jesse Coultas, Joseph Wiseman, and Luís Pina. 2026. Artifact for PropCov: Effective Coverage Reporting for Property-Based Testing. <https://zenodo.org/records/17095525>. <https://doi.org/10.5281/zenodo.17095525>
- [28] Jesse Coultas, Joseph Wiseman, and Luís Pina. 2026. PropCov Repository. <https://github.com/bitslab/propcov-public>.
- [29] Colin Dellow. 2017. manu: A time series storage format for integers and floats, using efficient delta encodings from FastPFOR. <https://github.com/cldellow/manu>. Accessed: 2026-07-13.
- [30] Michael Eichberg and Ben Hermann. 2014. A software product line for static analyses: the OPAL framework. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis* (Edinburgh,

- United Kingdom) (SOAP '14). Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/2614628.2614630>
- [31] M. Eichberg, F. Kübler, D. Helm, M. Reif, G. Salvaneschi, and M. Mezini. 2018. Lattice based modularization of static analyses. In *Companion Proceedings for the ISSTA/ECOOP 2018 Workshops* (Amsterdam, Netherlands) (ISSTA '18). Association for Computing Machinery, New York, NY, USA, 113–118. <https://doi.org/10.1145/3236454.3236509>
 - [32] Sebastian Elbaum, David Gable, and Gregg Rothermel. 2001. The Impact of Software Evolution on Code Coverage Information. In *Proceedings of the IEEE International Conference on Software Maintenance (ICSM'01)* (ICSM '01). IEEE Computer Society, USA, 170. <https://doi.org/10.1109/ICSM.2001.972727>
 - [33] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++ : Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association. <https://www.usenix.org/conference/woot20/presentation/fioraldi>
 - [34] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2013. Comparing non-adequate test suites using coverage criteria. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) (ISSTA 2013). Association for Computing Machinery, New York, NY, USA, 302–313. <https://doi.org/10.1145/2483760.2483769>
 - [35] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. *SIGPLAN Not.* 40, 6 (June 2005), 213–223. <https://doi.org/10.1145/1064978.1065036>
 - [36] Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. 2024. Property-Based Testing in Practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 187, 13 pages. <https://doi.org/10.1145/3597503.3639581>
 - [37] Harrison Goldstein, Jeffrey Tao, Zac Hatfield-Dodds, Benjamin C. Pierce, and Andrew Head. 2024. Tyche: Making Sense of PBT Effectiveness. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, UIST 2024, Pittsburgh, PA, USA, October 13-16, 2024*, Lining Yao, Mayank Goel, Alexandra Ion, and Pedro Lopes (Eds.). ACM, 10:1–10:16. <https://doi.org/10.1145/3654777.3676407>
 - [38] Gradle Inc. 2008. Gradle Build Tool. <https://gradle.org/>. Accessed: 2026-07-13.
 - [39] Christopher M. Hayden, Edward K. Smith, Eric A. Hardisty, Michael Hicks, and Jeffrey S. Foster. 2012. Evaluating Dynamic Software Update Safety Using Systematic Testing. *IEEE Trans. Softw. Eng.* 38, 6 (Nov. 2012), 1340–1354. <https://doi.org/10.1109/TSE.2011.101>
 - [40] Dominik Helm, Tobias Roth, Sven Keidel, Michael Reif, and Mira Mezini. 2024. Unimocg: Modular Call-Graph Algorithms for Consistent Handling of Language Features. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 51–62. <https://doi.org/10.1145/3650212.3652109>
 - [41] Michael Hilton, Jonathan Bell, and Darko Marinov. 2018. A large-scale study of test coverage evolution. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering* (Montpellier, France) (ASE '18). Association for Computing Machinery, New York, NY, USA, 53–63. <https://doi.org/10.1145/3238147.3238183>
 - [42] Marc R. Hoffmann. 2009. Eclemma - JaCoCo Java Code Coverage Library. <https://www.eclemma.org/jacoco/>. Accessed: 2026-07-05.
 - [43] Paul Holser. 2023. Property-based testing, JUnit-style. <https://github.com/pholser/junit-quickcheck>. Accessed: 2026-07-05.
 - [44] Indeed Corp. 2018. Indeed MPH: Fast and Compact Immutable Key-Value Stores. <https://engineering.indeedblog.com/blog/2018/02/indeed-mph/>. Accessed: 2026-07-05.
 - [45] LLVM Compiler Infrastructure. 2016. libFuzzer - a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer>. Accessed: 2026-07-05.
 - [46] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). Association for Computing Machinery, New York, NY, USA, 2123–2138. <https://doi.org/10.1145/3243734.3243804>
 - [47] James Kukucka, Luís Pina, Paul Ammann, and Jonathan Bell. 2022. CONFETTI: Amplifying Concolic Guidance for Fuzzers. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 438–450. <https://doi.org/10.1145/3510003.3510628>
 - [48] Wing Lam, Siwakorn Srisakaokul, Blake Bassett, Peyman Mahdian, Tao Xie, Pratap Lakshman, and Jonathan de Halleux. 2018. A Characteristic Study of Parameterized Unit Tests in .NET Open Source Projects. In *32nd European Conference on Object-Oriented Programming (ECOOP 2018)* (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 109), Todd Millstein (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 5:1–5:27. <https://doi.org/10.4230/LIPIcs.ECOOP.2018.5>
 - [49] Danushka Liyanage, Marcel Böhme, Chakkrit Tantithamthavorn, and Stephan Lipp. 2023. Reachable Coverage: Estimating Saturation in Fuzzing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*.

- 371–383. <https://doi.org/10.1109/ICSE48619.2023.00042>
- [50] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (Hong Kong, China) (FSE 2014)*. Association for Computing Machinery, New York, NY, USA, 643–653. <https://doi.org/10.1145/2635868.2635920>
- [51] Paul Marinescu, Petr Hosek, and Cristian Cadar. 2014. Covrig: a framework for the analysis of code, test, and coverage evolution in real software. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 93–104. <https://doi.org/10.1145/2610384.2610419>
- [52] Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. An Empirical Study of the Reliability of UNIX Utilities. *Commun. ACM* 33, 12 (dec 1990), 32–44. <https://doi.org/10.1145/96267.96279>
- [53] Gerard Murphy. 2022. Americium: Generation of test case data for Scala and Java, in the spirit of QuickCheck. <https://github.com/sageserpent-open/mericium>. Accessed: 2026-07-13.
- [54] Rickard Nilsson. 2023. ScalaCheck: Property-based testing for Scala. <https://scalacheck.org/>. Accessed: 2026-07-05.
- [55] Robert O’Callahan, Chris Jones, Nathan Froyd, Kyle Huey, Albert Noll, and Nimrod Partush. 2017. Engineering record and replay for deployability. In *Proceedings of the 2017 USENIX Conference on Usenix Annual Technical Conference (Santa Clara, CA, USA) (USENIX ATC ’17)*. USENIX Association, USA, 377–389.
- [56] Jesper Olsson. 2017. capsyl: Visualizes Java objects’ encapsulations. <https://github.com/jesperolsson-se/capsyl>. Accessed: 2026-07-13.
- [57] Opal Project. 2021. OPAL Documentation - Facilitating the Development of Static Analyses. <https://github.com/opal/j/opal/blob/488ec8d4962aeb7cfa9ba2e4af70bfe60c7d50f2/src/site/DeveloperTools.md?plain=1#L72>. Accessed: 2026-07-13.
- [58] Rohan Padhye, Caroline Lemieux, and Koushik Sen. 2019. JQF: Coverage-Guided Property-Based Testing in Java. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 398–401. <https://doi.org/10.1145/3293882.3339002>
- [59] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 329–340. <https://doi.org/10.1145/3293882.3330576>
- [60] Luís Pina and Michael Hicks. 2016. Tedsuto: A General Framework for Testing Dynamic Software Updates. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 278–287. <https://doi.org/10.1109/ICST.2016.27>
- [61] Michael Reif, Michael Eichberg, Ben Hermann, Johannes Lerch, and Mira Mezini. 2016. Call graph construction for Java libraries. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Seattle, WA, USA) (FSE 2016)*. Association for Computing Machinery, New York, NY, USA, 474–486. <https://doi.org/10.1145/2950290.2950312>
- [62] John Rose, Ola Bini, William R Cook, Rémi Forax, Samuele Pedroni, and Jochen Theodorou. 2011. JSR 292: Supporting Dynamically Typed Languages on the JavaTM Platform. <https://jcp.org/ja/jsr/detail?id=292>. Accessed: 2026-07-13.
- [63] David Schwartz, Ankith Kowshik, and Luís Pina. 2024. Jmvx: Fast Multi-threaded Multi-version Execution and Record-Replay for Managed Languages. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 329 (Oct. 2024), 29 pages. <https://doi.org/10.1145/3689769>
- [64] David Schwartz and Luís Pina. 2026. Optimizing Record/Replay Through Relaxed Total Ordering and Multi-Version eXecution. In *40th European Conference on Object-Oriented Programming (ECOOP 2026) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 372)*, Robbert Krebbers and Alexandra Silva (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 24:1–24:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2026.24>
- [65] Manu Sridharan, Stephen J. Fink, and Rastislav Bodik. 2007. Thin slicing. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (San Diego, California, USA) (PLDI ’07)*. Association for Computing Machinery, New York, NY, USA, 112–122. <https://doi.org/10.1145/1250734.1250748>
- [66] Nikolai Tillmann and Wolfram Schulte. 2005. Parameterized unit tests. *SIGSOFT Softw. Eng. Notes* 30, 5 (Sept. 2005), 253–262. <https://doi.org/10.1145/1095430.1081749>
- [67] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundareshan. 2010. Soot: a Java bytecode optimization framework. In *CASCON First Decade High Impact Papers (Toronto, Ontario, Canada) (CASCON ’10)*. IBM Corp., USA, 214–224. <https://doi.org/10.1145/1925805.1925818>
- [68] Cindy Wauters and Coen De Roover. 2024. Property-based Testing within ML Projects: an Empirical Study. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 648–653. <https://doi.org/10.1109/ICSME58944.2024.00067>
- [69] Cindy Wauters, Ruben Opdebeeck, and Coen De Roover. 2026. On the Evolution of Python Test Cases into Property-based Tests. In *2026 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 429–440. <https://doi.org/10.1109/ICST69053.2026.00062>

- [70] Mark Weiser. 1984. Program Slicing. *IEEE Trans. Softw. Eng.* 10, 4 (July 1984), 352–357. <https://doi.org/10.1109/TSE.1984.5010248>
- [71] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. *SIGPLAN Not.* 46, 6 (jun 2011), 283–294. <https://doi.org/10.1145/1993316.1993532>
- [72] Michał Zalewski. 2014. American Fuzzy Lop. <https://lcamtuf.coredump.cx/afl/>. Accessed: 2026-07-05.

Received 2026-01-29; accepted 2026-04-16